

LoRA (Low-Rank Adaptation of Large Language Models)

Revision Guide

Beginner-friendly revision notes based on our study sessions.

Why LoRA?

Full fine-tuning updates every parameter in a huge LLM, making training and storage extremely expensive.

Core Idea

Freeze the pretrained model and train only a tiny update ΔW .

Main Formula

Output = $(W + \Delta W)x$ where $\Delta W = BA$. A and B are much smaller matrices than W.

Low Rank

The adaptation needed for new tasks is much simpler than the original model, so a low-rank update is often enough.

Forward Pass

$h = Wx + BAx$. W stays frozen. Only A and B learn.

Initialization

A is random, B is zero, so $BA=0$ initially and the model starts identical to the pretrained model.

Scaling

The update is scaled by α/r to keep learning stable across different rank values.

Transformer Layers

LoRA is mainly applied to the Query (W_q) and Value (W_v) projection matrices in the paper.

No Inference Overhead

After training, merge BA into W . The deployed model runs like a normal model with no extra latency.

Experimental Results

Comparable performance to full fine-tuning while using far fewer trainable parameters, much lower memory, and tiny checkpoints.

Advantages

Less GPU memory, tiny checkpoints, fast task switching, efficient fine-tuning.

Limitations

Choice of rank matters; paper mainly studies adaptation of Transformer linear layers.

Complete Workflow

Pretrained Model → Freeze Weights → Add A & B → Train Only A & B → Merge BA into W → Deploy.

Interview Questions

Why is full fine-tuning expensive? Explain $\Delta W = BA$. What is low rank? Why W_q and W_v ? Why merge BA into W?

One-Line Summary

LoRA freezes the original LLM and learns a tiny low-rank update ($\Delta W = BA$), achieving efficient fine-tuning with drastically lower memory and no extra inference cost after merging.